



SDB: Safety Constraint Mechanism for Dual-Branch End-to-End Autonomous Driving

Hongzhi Yan  and Jianmei Su  

Southwest University of Science and Technology, Mianyang, Sichuan, China
sujm_swust@163.com

Abstract. Autonomous driving requires predicting a safe and compliant trajectory to facilitate reliable driving decisions. Previous end-to-end autonomous driving approaches usually research in two directions, one is based on the planned trajectory, and the other is making control decisions directly. In this paper, the proposed autonomous driving model is a two-branch combining architecture. One is waypoint prediction branch, which uses Multilayer Perceptron(MLP) prediction to generate a sequence of future waypoints to obtain a safe and reliable trajectory by using three penalty constraint. The other branch is the Proportion Integration Differentiation(PID) controller, which dynamically adjusts the decision-making control command relying on the vehicle state and trajectory point and optimal waypoints prediction branch. The experimental evaluation on the CARLA simulator shows that the proposed approach achieves the best performance on closed-loop benchmarks over state of the art approaches.

Keywords: End-to-End · Dual-Branch Structure · Dynamic BEV Feature Fusion · Penalty Constraint

1 Introduction

With the rapid development of deep learning in recent years, autonomous driving has experienced a shift from modular [20,26], handcrafted pipelines to integrated end-to-end paradigms [11–13,25]. The end-to-end framework emphasizes the concept of seamless integration of perception, decision making and control, which has gradually become a research hotspot of autonomous driving. However, there are still challenges in feasibility and security for the end-to-end planning control method. There are two main reasons: (1) the control output is susceptible to local disturbances due to the lack of forward-looking prediction and constraints on the future trajectory. (2) The lack of effective decoupling between control decision and trajectory planning makes it difficult to achieve real-time collaboration in dynamic scenarios.

For the above problems, many researchers have proposed solutions. Intermediate representations are used as auxiliary supervision signals to enhance the

stability and generalization of the model. Bansal et al. [1] used trajectory prediction as an intermediate representation to model future trajectory generation and guide control strategies. However, early methods only generated a single trajectory, which could not cover multiple driving behaviors. Varadarajan et al. [22] addressed the issue of insufficient multi-modal coverage by introducing probabilistic anchors to generate multi-modal trajectory distributions and improve the rationality of trajectories in complex intersection scenarios. Nevertheless, traditional trajectory prediction modules often ignore compliance and safety constraints on the predicted trajectories, leading to unreasonable driving behaviors such as overlapping with other agents, exceeding the drivable area, and driving in the wrong direction.

To solve this problem, this paper proposes a Safety Constraint Mechanism for Dual-Branch End-to-End Autonomous Driving (SDB). To better integrate high-level trajectory planning with low-level motion control, a dual-branch architecture is proposed. One branch performs waypoint prediction and is used to guide the decision-making process of the other branch, enabling the model to utilize the predicted waypoint information as prior knowledge for safer and more compliant driving behavior.

In the encoder part, a gated dynamic Bird's Eye View(BEV) feature fusion method is adopted, which can adaptively fuse different modal features to improve the robustness of the model. The decoder part, multiple penalty constraints are introduced in the trajectory prediction process: Collision Penalty Constraint, Boundary Penalty Constraint and Directional Penalty Constraint. As shown in Fig. 2. Mainly to ensure the safety and compliance of the self vehicle in the process of driving. To better weight the three penalty constraint loss functions, we use the weight adaptive allocation method to dynamically adjust the weights of the three loss functions. It avoids the difficulty of manually adjusting the weights. This method has been verified on the simulation platform CARLA [9] and achieved excellent performance.

2 Related Work

End-to-End Autonomous Driving. The traditional modular paradigm has the severe disadvantages of information error accumulation and high maintenance cost. Therefore, in recent years, the end-to-end [10, 11, 13, 25] paradigm of autonomous driving is a hot research issue. End-to-End paradigm raw sensor input can be divided into single-modal and multi-modal methods. However, the single-modal method has limited generalization ability and lacks the comprehensiveness of scene understanding. Compared with the single-modal method, the multi-modal method complements the disadvantages between different modes and has good robustness in complex or extreme scenes.

In multi-modal fusion, Vora et al. [23] append image semantic information to the point cloud. Wang et al. [24] enhance point cloud representations using image features. Chitta et al. [7] perform multi-scale feature fusion of RGB camera and LiDAR input information through a Transformer architecture. Liu et

al. [17] fuse multiple modalities using a unified BEV space. Jiang et al. [13] proposed a vectorization-based, end-to-end autonomous driving model that represents the driving scene as a fully vectorized format, predicts future trajectories, and enhances the robustness of scene understanding. Zhang et al. [28] adopt an end-to-end paradigm, directly outputting the low-level control signals, including steering angle, throttle, and brake. These works promote the development of the end-to-end paradigm of autonomous driving, improve the task efficiency and scene adaptation ability of autonomous driving, and reveal future technical challenges and optimization directions.

BEV Representation in Autonomous Driving. Bird’s Eye View (BEV) represents its core advantages in terms of unified 3D spatial representation, robustness to occlusion, and natural fit with downstream tasks (planning and control). Lang et al. [15] proposed a point cloud coding method based on pillars, which laid the foundation for real-time LiDAR detection. With the development of multi-modal perception, effective projection of image information from the camera perspective to BEV space has become a new research hotspot. Some methods choose to build BEV representations of images directly based on geometric projections. Phillion et al. [18] use the estimated depth map to project image features into BEV space and perform spatial fusion. Hu et al. [10] proposed to use multiple frames of time-series images to extract BEV features to realize time-series modeling and future prediction.

There are also transformer-based methods [16] that interact between image features and BEV queries through a spatial cross-attention mechanism to obtain high-quality BEV representations without accurate depth estimation. Chen et al. [5] convert front-view features into BEV space using Deformable Attention, effectively addressing the geometric distortion caused by perspective projection and improving the accuracy of lane line localization in 3D space. The BEV representation also demonstrates great potential in planning and decision-making. Hu et al. [11] proposed the first BEV-based end-to-end framework that jointly optimizes perception, prediction, and planning tasks, enabling the system to complete the entire autonomous driving task directly based on the BEV representation. This work provides a valuable reference and inspiration for the development of end-to-end autonomous driving technologies.

3 Method

Overview. The overall framework of SDB is shown in Fig. 1. Image encoder (ResNet-34) and LiDAR encoder (ResNet-18) are used to extract image feature F_{img} and LiDAR feature F_{LiDAR} for RGB images and LiDAR images, respectively. In the process of extracting F_{img} , the LSS [18] algorithm is used to predict the depth distribution of image pixels and generate 3D feature point cloud along the line of sight direction, and then the point cloud is projected into the BEV grid for feature feature aggregation to generate RGB image feature $F_{\text{i,BEV}}$ in the form of BEV. Following the practice of SECOND [27], the LiDAR encoder

applies a sparse 3D convolution to the voxelized point cloud, and finally also outputs the feature $F_{\text{LiDAR-BEV}}$ of the laser LiDAR modality in the form of BEV. In the feature fusion part, we dynamically fuse two BEV form features (paragraph 3.1).

The fused BEV feature $F_{\text{BEV}} \in \mathbb{R}^{B \times 256 \times 21 \times 21}$ will be spliced with the feature vector $F_c \in \mathbb{R}^{B \times 128}$ of the self-vehicle state information encoded by MLP to generate the fused feature $F_{\text{out}} \in \mathbb{R}^{B \times 384}$ for decision-making, which is used in the decoder for direct control branch and waypoint prediction branch. Among them, the direct control branch directly predicts the throttle value, brake value, steering value and other decision quantities. The time series information in the future position information generated by the waypoint prediction branch can make the direct control branch consider the future motion intention of the surrounding agents in the decision-making process, so as to plan in advance and avoid potential dangers. In addition, traffic constraint information such as traffic lights and stop signs will be used as additional penalty items to constraint the future path generated by the self-vehicle. When the predicted future path conflicts with traffic rules, the model will be penalized, so as to ensure that the decisions generated by the model comply with traffic rules and improve the safety of decision-making.

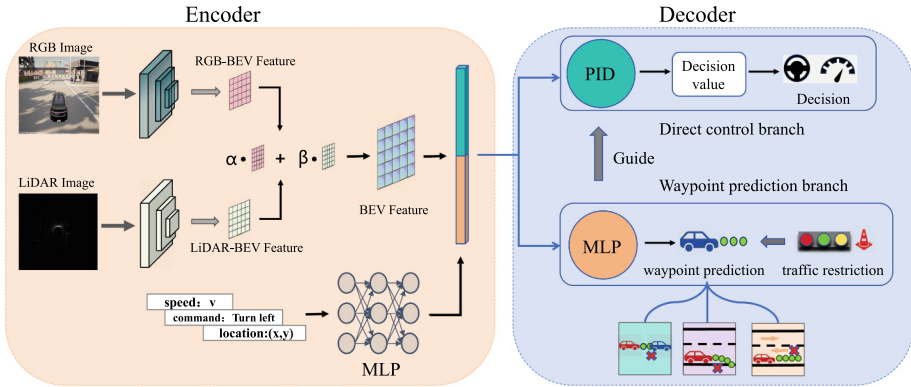


Fig. 1. Overall architecture of SDB. The overall SDB framework consists of two parts: encoder and decoder. The encoder processes images from various sensors and converts them into a Bird's Eye View (BEV), which serves as input for the decoder. The decoder consists of a direct control branch and a waypoint prediction branch, with the future trajectory information generated by the waypoint prediction branch guiding the final decision of the direct control branch.

3.1 Dynamic BEV Feature Fusion

In this study, dynamic BEV feature fusion is a multi-modal feature fusion method based on dynamic gating mechanism, which dynamically weight fuses LiDAR

features in the form of BEV and RGB camera features. This fusion mechanism fully complements the advantages of both. It can also adaptively assign modal weights to the importance of input features according to the scene environmental factors. Compared with fixed weight or simple concatenation, it can effectively deal with the difference of modal feature distribution and local noise interference. For the input RGB image I_{img} and LiDAR image I_{LiDAR} , ResNet-34 and ResNet-18 were used to extract features F_{img} and F_{LiDAR} , respectively. The specific steps are as follows. First, the two features are transformed by 1×1 convolution:

$$\tilde{F}_{\text{img}} = C_{1 \times 1}(F_{\text{img}}), \quad \tilde{F}_{\text{LiDAR}} = C_{1 \times 1}(F_{\text{LiDAR}}) \quad (1)$$

where $C_{1 \times 1}$ is 1×1 convolution, in this way, the expression ability of features can be improved while keeping the spatial size unchanged, for better fusion in the future. Image features and LiDAR features are concatenated to generate gating weights:

$$F_{\text{concat}} = \text{Concat}(\tilde{F}_{\text{img}}, \tilde{F}_{\text{LiDAR}}) \in \mathbb{R}^{B \times 2C \times H \times W} \quad (2)$$

$$G = \sigma(C_{1 \times 1}(\text{ReLU}(C_{3 \times 3}(F_{\text{concat}})))) \in \mathbb{R}^{B \times 2 \times H \times W} \quad (3)$$

where σ is the Sigmoid function, $C_{3 \times 3}$ is the 3×3 convolution, and G is the modal weight after fusion. The concatenated feature F_{concat} contains the joint information of the two modalities in space. Complete context information can be provided to the gated network. The subsequent gating network will output the fusion weights of the two channels, which can select the importance of image or LiDAR features on demand, so as to achieve high-quality modal fusion. After that, the fusion weight G is weight separated:

$$\alpha = G[:, 0, :] \quad \beta = G[:, 1, :] \quad (4)$$

where α is the image feature weight, β is the LiDAR feature weight. Using Softmax to ensure weight normalization, that is:

$$\alpha + \beta = 1 \quad (5)$$

Finally, the RGB features and LiDAR features in BEV format are dynamically weighted fused, and the final residual connection output:

$$F_{\text{fused}} = \alpha \cdot F_{\text{LiDAR}} + \beta \cdot F_{\text{img}} \quad (6)$$

$$F_{\text{BEV}} = F_{\text{fused}} + F_{\text{img}} + F_{\text{LiDAR}} \quad (7)$$

Each pixel uses the adaptive weight to perform the weighted sum of the two modal features, so as to achieve dynamic fusion. The final double residual link mainly retained the feature information in the image modality and LiDAR modality to prevent the loss of detail information after fusion. Secondly, it can also slow down the vanishing gradient and increase the stability of training.

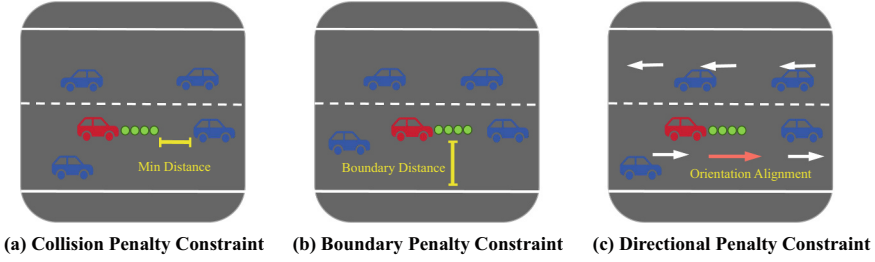


Fig. 2. Illustration of Penalty Constraint. Collision penalty constraints ensure the safety of autonomous vehicles and other intelligent entities. Boundary penalty constraints prevent future waypoints from approaching or exceeding boundaries. Directional penalty constraints ensure that the driving direction of the autonomous vehicle aligns with the lane direction.

3.2 Collision Penalty Constraint

As shown in Fig. 2 (a). The Collision Penalty Constraint method in this study designs an exponential penalty term based on the minimum distance between the sequence of waypoints $W_{t+1}, W_{t+2}, \dots, W_{t+T} \in \mathbb{R}^{B \times T \times 2}$ generated by the model prediction (2 represents the plane coordinates (x,y) of each point) and the agent. Specifically, first SDB calculates the Euclidean distance using the predicted sequence of waypoints and the 2D coordinate positions of other dynamic and static objects:

$$D_{b,t,n} = \|W_t - O_n\|_2 \quad t \in [1, T], n \in [1, N] \quad (8)$$

where b is the sample batch index. t is the waypoint time series index. n is the number of agents in the scenario. Then for the waypoint, we take the distance to the closest agent:

$$d_{b,t} = \min_n D_{b,t,n} \quad (9)$$

Finally, the penalty constraint loss is calculated:

$$\mathcal{L}_{\text{collision}} = \frac{1}{B \cdot T} \sum_{b=1}^B \sum_{t=1}^T \exp\left(-\frac{d_{bt}}{\delta}\right) \quad (10)$$

where $\exp\left(-\frac{d_{bt}}{\delta}\right)$ is the penalty term, that is, the loss is larger for closer distances and smaller for the opposite. δ is the penalty parameter (default 1.0). Finally, a scalar average penalty constraint loss is derived. The mainly penalizes the behavior of predicting waypoints close to other agents to improve the safety of autonomous driving planning and avoid collisions.

3.3 Boundary Penalty Constraint

As shown in Fig. 2 (b). The Boundary Penalty Constraint method in SDB mainly penalizes the behavior of the predicted waypoint approaching the boundary to avoid the unreasonable risk of falling on the road boundary or crossing the boundary. Specifically, Since the predicted waypoints are the real world coordinates in the actual scene, we need to look up the position value on the BEV feature map, so the real world waypoints are first mapped to the pixel coordinate system on the BEV feature map, and then the horizontal/vertical coordinates of each waypoint position are indexed:

$$(x_{b,t}, y_{b,t}) = \text{clamp}(\lfloor s \cdot W_{b,t} \rfloor, 0, H - 1) \quad (11)$$

where $\lfloor \cdot \rfloor$ denotes the discrete grid coordinates of integers to BEV, and the floating-point coordinates are transformed into integer indexes; s is the scaling factor (default 0.5); H is the size of the BEV feature map; $\text{clamp}(j, k, l)$ is a limiting function, limiting j to the $[k, l]$ interval mainly prevents the index from crossing the boundary. Find the value of the BEV boundary mask:

$$M_{b,t} = F_{\text{BEV}}(b, c, x_{b,t}, y_{b,t}) \quad (12)$$

where c is the boundary mask channel. For the waypoints are provided with how far or near the point is from the drivable area in the current BEV feature map. The margin penalty is calculated using the soft-margin strategy:

$$\mathcal{L}_{b,t} = \max(0, \text{margin} - M_{b,t}) \quad (13)$$

where margin is a security threshold (default 0.2). When the boundary mask value is less than the margin, the penalty will be generated, and the closer to the boundary, the greater the penalty. Finally, the loss of all batch and time steps is averaged to obtain the overall boundary constraint loss:

$$\mathcal{L}_{\text{bound}} = \frac{1}{B \cdot T} \sum_{b=1}^B \sum_{t=1}^T \mathcal{L}_{b,t} \quad (14)$$

This penalty mechanism aggregates the boundary penalties of all waypoints, penalizes the predicted trajectory to be close to the boundary, and enhances the robustness of the model.

3.4 Directional Penalty Constraint

As shown in Fig. 2 (c). The purpose of the Directional Penalty Constraint in this study is to make the predicted trajectory direction consistent with the geometric direction of the road, so as to ensure that the autonomous vehicle's driving trajectory conforms to the road topology (such as lane lines and lane directions). Specifically, we first obtain the direction vector of the predicted path points in the BEV feature map (the difference between adjacent path points in the physical space):

$$\mathbf{v}_{b,t} = W_{b,t+1} - W_{b,t}, \quad t = 0, 1, \dots, T-2 \quad (15)$$

A total of $T-1$ direction vectors were obtained, after which lane direction vectors were extracted from BEV feature maps and vector normalization was performed:

$$\mathbf{d}_{b,t} = (d_{b,t}^x, d_{b,t}^y) \quad (16)$$

$$v_{b,t} = \frac{\mathbf{v}_{b,t}}{\|\mathbf{v}_{b,t}\|} \quad d_{b,t} = \frac{\mathbf{d}_{b,t}}{\|\mathbf{d}_{b,t}\|} \quad (17)$$

Both the direction of the predicted point and the direction of the lane line are converted into unit vectors. The cosine similarity is then used to calculate the alignment of the two directions:

$$\cos(\theta_{b,t}) = v_{b,t} \cdot d_{b,t} \quad (18)$$

where $\cos(\theta_{b,t}) \in [-1, 1]$. A higher value indicates a more consistent direction, otherwise the opposite is true. Finally, the directional loss is averaged over all trajectory segments:

$$\mathcal{L}_{\text{dir}} = \frac{1}{B(T-1)} \sum_{b=1}^B \sum_{t=1}^T (1 - \cos(\theta_{b,t})) \quad (19)$$

When the direction is consistent, $\cos(\theta_{b,t}) = 1$, the loss is 0; when $\cos(\theta_{b,t}) = -1$ in the opposite direction, the maximum loss is 2. This penalty constraint method improves the direction compliance of the waypoints generated by the model and avoids violations such as retrograde.

3.5 End-to-End Loss

For the three penalty constraint losses in the waypoint prediction branch. Inspired by [14], we adaptively assign three kinds of loss weights. Specifically, the loss weight of each task is automatically adjusted by modeling the uncertainty of the task itself. We first initialize learnable weights, then forward compute the loss for each constraint, and automatically sum up the individual loss terms:

$$\mathcal{L}_{\text{plan}} = \sum_i \frac{1}{2\sigma_i^2} \mathcal{L}_i + \frac{1}{2} \log \sigma_i^2 \quad \{i \in \{\mathcal{L}_{\text{collision}}, \mathcal{L}_{\text{bound}}, \mathcal{L}_{\text{dir}}\}\} \quad (20)$$

where σ_i is a learnable parameter that's automatically optimized through back-propagation. $\log \sigma_i$ is the regularization term. $\frac{1}{2\sigma_i^2}$ is the task weight. $\mathcal{L}_{\text{plan}}$ is the waypoint prediction branch loss. For the direct control branch, the loss mainly comes from the error between the control signal output by the model (speed, throttle, steering, etc.) and the real control signal, where the control signal error is calculated by L1 loss. For the total loss, there is also a multi-task auxiliary loss,

which is mainly to enhance the quality of BEV representation, traffic rule understanding, feature consistency prediction and other auxiliary goals. Although it does not directly affect the control output, it is of great help to the stability and generalization of model training. That is, the total end-to-end loss is as follows:

$$\mathcal{L} = \omega_1 \mathcal{L}_{\text{plan}} + \omega_2 \mathcal{L}_{\text{cls}} + \omega_3 \mathcal{L}_{\text{aux}} \quad (21)$$

where $\omega_1, \omega_2, \omega_3$ are weight coefficients. $\mathcal{L}_{\text{cls}}$ is to control the branch loss directly. $\mathcal{L}_{\text{aux}}$ is the multi-task auxiliary loss.

4 Experiment

In this section, our method is tested in the validation of CARLA (0.9.10) driving simulator [9], using three metrics to evaluate the performance of our model: The Route Completion (RC) is used to measure the ability of the vehicle to successfully complete the specified route. Infraction Score (IS) reflects the degree of compliance of vehicles with traffic rules. Driving Score (DS), as a comprehensive performance index, is the main measurement standard, reflecting the comprehensive ability of the model to complete the route and obey the traffic rules in the driving task. And we deploy agent vehicles in 8 towns with different weather conditions of CARLA, and collect 600 000 frames of data at 10 frames per second as the dataset for model training to provide diverse and representative reference samples for model learning.

Table 1. Performance on Longest6 benchmark

RANK	Method	Driving Score $\uparrow$	Route Completion $\uparrow$	Infraction Score $\uparrow$
1	InterFuser [21]	38.177	73.179	0.526
2	Latent TransFuser* [7]	37.31	95.18	0.38
3	LAV* [3]	32.74	70.36	0.51
4	WOR* [2]	20.53	48.47	0.56
5	SDB(Ours)	46.859	71.485	0.692

4.1 Comparative Experiment

In this study, a number of comparative experiments are conducted to compare and analyze the performance of various existing automated driving decision-making methods with excellent performance in two benchmark tests, Longest6 and Town05Long. Using the symbols ‘*’ or ‘ $\uparrow$ ’ denotes experimental results from TransFuser [19] and InterFuser [21], respectively. The data labeled as ‘—’ in the violation score column, indicates that the relevant literature does not provide specific results under the benchmark. The test index of ‘ $\uparrow$ ’ indicates that the

Table 2. Performance on Town05 Long benchmark

RANK	Method	Driving Score $\uparrow$	Route Completion $\uparrow$	Infraction Score $\uparrow$
1	InterFuser [21]	50.664	84.934	0.614
2	WOR* [2]	44.80	82.41	—
3	Roach* [28]	43.64	80.37	—
4	NEAT* [6]	37.72	62.13	—
5	TransFuser* [19]	33.15	56.36	—
6	LBC* [4]	7.05	32.09	—
7	CILRS* [8]	3.68	7.19	—
8	ST-P3 [10]	11.45	83.15	—
9	VAD-Base [13]	30.31	75.20	—
10	SDB(Ours)	56.745	81.953	0.708

higher the value, the better the performance. The bold result indicates that this method has achieved the best performance among the current test methods.

As shown in Table 1 and 2, our model has the highest driving score and infraction score in the autonomous driving task. In the infraction score (IS) indicator, our model shows excellent driving safety and compliance. Additionally, Comprehensive indicator driving score (DS) has also achieved optimal performance among many methods.

We also verify its robustness under different weather conditions, and compare and analyze the performance in a variety of scenarios, including night, heavy rain and sunny weather. As shown in Table 3. Comprehensive validation has demonstrated that SDB exhibits strong robustness under various complex weather conditions, while also achieving a certain degree of performance improvement.

In the comparative experiments, although our model did not achieve the best performance in the route completion (RC) metric, this precisely reflects our emphasis on safety and compliance with traffic rules. In the future, while maintaining high safety standards, we will further strive to improve route completion to achieve more comprehensive driving performance.

As shown in Fig 3. We present the visualizations of SDB under different weather conditions.

4.2 Ablation Studies

We set up a series of ablation implementations that verify the effectiveness of different component designs of our model. Where ‘col’, ‘bound’, and ‘dir’ represent Collision Penalty Constraint, Boundary Penalty Constraint, and Directional Penalty Constraint, respectively. ‘DF’ stands for dynamic BEV feature fusion.

As shown in Table 4, our designed three penalty mechanisms and dynamic BEV feature fusion have positive improvement for autonomous driving task performance.

Table 3. Performance of SDB under different weather conditions

Weather	Algorithm	Driving Score $\uparrow$	Route Completion $\uparrow$	Infraction Score $\uparrow$
CloudyNight	InterFuser	39.070	74.982	0.512
	SDB(Ours)	46.108	71.251	0.623
HardrainNoon	InterFuser	34.775	73.483	0.481
	SDB(Ours)	43.512	71.204	0.642
MidrainMorning	InterFuser	40.105	73.133	0.580
	SDB(Ours)	47.073	72.114	0.705

Table 4. Result of Ablation Studies

Algorithm	Driving Score $\uparrow$	Route Completion $\uparrow$	Infraction Score $\uparrow$
w/o col&bound&dir&DF	37.849	61.263	0.603
w/o col&bound&dir	38.637	61.896	0.611
w/o bound&dir&DF	40.974	66.548	0.641
w/o col&dir&DF	41.346	68.523	0.653
w/o col&bound&DF	40.052	66.768	0.638
SDB	46.859	71.485	0.692

**Fig. 3.** Visualization of different weather conditions

5 Conclusion

In this work, we propose a Safety Constraint Mechanism for Dual-Branch End-to-End Autonomous Driving. In the feature fusion stage, we employ a dynamic fusion method that combines image features in BEV format with LiDAR features. The advantage of this approach is its ability to dynamically and adap-

tively accommodate multi-modal data fusion, thereby improving robustness. For the waypoint prediction branch, we propose three penalty constraints: Collision Penalty Constraint; Boundary Penalty Constraint; Directional Penalty Constraint. These three penalty constraints improve the safety and compliance of the predicted trajectory path of the model. Finally, the overall framework adopted a dual-branch structure, and the predicted future trajectory of the waypoint predicted branch output was used to guide the decision-making process of the direct control branch. In this way, the error problem of path prediction is avoided and the lack of time series information in direct control branch species is complemented. Thus, more accurate decision making and safer driving behavior can be achieved.

Acknowledgments. This paper is supported by the Major Project of Sichuan Provincial Natural Science Foundation (2025ZNSFSC0005), and the Central Government Guides Local Projects of China (2025ZYDF105).

References

1. Bansal, M., Krizhevsky, A., Ogale, A.: Chauffeurnet: learning to drive by imitating the best and synthesizing the worst. arXiv preprint [arXiv:1812.03079](https://arxiv.org/abs/1812.03079) (2018)
2. Chen, D., Koltun, V., Krähenbühl, P.: Learning to drive from a world on rails. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 15590–15599 (2021)
3. Chen, D., Krähenbühl, P.: Learning from all vehicles. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 17222–17231 (2022)
4. Chen, D., Zhou, B., Koltun, V., Krähenbühl, P.: Learning by cheating. In: Conference on Robot Learning, pp. 66–75. PMLR (2020)
5. Chen, L., et al.: Persformer: 3d lane detection via perspective transformer and the openlane benchmark. In: European Conference on Computer Vision, pp. 550–567. Springer, Heidelberg (2022)
6. Chitta, K., Prakash, A., Geiger, A.: Neat: neural attention fields for end-to-end autonomous driving. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 15793–15803 (2021)
7. Chitta, K., Prakash, A., Jaeger, B., Yu, Z., Renz, K., Geiger, A.: Transfuser: imitation with transformer-based sensor fusion for autonomous driving. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(11), 12878–12895 (2022)
8. Codevilla, F., Santana, E., López, A.M., Gaidon, A.: Exploring the limitations of behavior cloning for autonomous driving. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 9329–9338 (2019)
9. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: Carla: an open urban driving simulator. In: Conference on Robot Learning, pp. 1–16. PMLR (2017)
10. Hu, S., Chen, L., Wu, P., Li, H., Yan, J., Tao, D.: St-p3: end-to-end vision-based autonomous driving via spatial-temporal feature learning. In: European Conference on Computer Vision, pp. 533–549. Springer, Heidelberg (2022). https://doi.org/10.1007/978-3-031-19839-7_31

11. Hu, Y., et al.: Planning-oriented autonomous driving. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 17853–17862 (2023)
12. Jia, X., et al.: Think twice before driving: towards scalable decoders for end-to-end autonomous driving. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 21983–21994 (2023)
13. Jiang, B., et al.: VAD: vectorized scene representation for efficient autonomous driving. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 8340–8350 (2023)
14. Kendall, A., Gal, Y., Cipolla, R.: Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7482–7491 (2018)
15. Lang, A.H., Vora, S., Caesar, H., Zhou, L., Yang, J., Beijbom, O.: Pointpillars: fast encoders for object detection from point clouds. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12697–12705 (2019)
16. Li, Z., et al.: Bevformer: learning bird’s-eye-view representation from lidar-camera via spatiotemporal transformers. *IEEE Trans. Pattern Anal. Mach. Intell.* (2024)
17. Liu, Z., et al.: Bevfusion: multi-task multi-sensor fusion with unified bird’s-eye view representation. In: 2023 IEEE International Conference on Robotics and Automation (ICRA), pp. 2774–2781. IEEE (2023)
18. Pillion, J., Fidler, S.: Lift, splat, shoot: encoding images from arbitrary camera rigs by implicitly unprojecting to 3D. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.-M. (eds.) ECCV 2020. LNCS, vol. 12359, pp. 194–210. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58568-6_12
19. Prakash, A., Chitta, K., Geiger, A.: Multi-modal fusion transformer for end-to-end autonomous driving. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7077–7087 (2021)
20. Sadat, A., Casas, S., Ren, M., Wu, X., Dhawan, P., Urtasun, R.: Perceive, predict, and plan: safe motion planning through interpretable semantic representations. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.-M. (eds.) ECCV 2020. LNCS, vol. 12368, pp. 414–430. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58592-1_25
21. Shao, H., Wang, L., Chen, R., Li, H., Liu, Y.: Safety-enhanced autonomous driving using interpretable sensor fusion transformer. In: Conference on Robot Learning, pp. 726–737. PMLR (2023)
22. Varadarajan, B., et al.: Multipath++: efficient information fusion and trajectory aggregation for behavior prediction. In: 2022 International Conference on Robotics and Automation (ICRA), pp. 7814–7821. IEEE (2022)
23. Vora, S., Lang, A.H., Helou, B., Beijbom, O.: Pointpainting: sequential fusion for 3d object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4604–4612 (2020)
24. Wang, C., Ma, C., Zhu, M., Yang, X.: Pointaugmenting: cross-modal augmentation for 3d object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11794–11803 (2021)
25. Wu, P., Jia, X., Chen, L., Yan, J., Li, H., Qiao, Y.: Trajectory-guided control prediction for end-to-end autonomous driving: a simple yet strong baseline. *Adv. Neural. Inf. Process. Syst.* **35**, 6119–6132 (2022)
26. Xu, W., Wang, Q., Dolan, J.M.: Autonomous vehicle motion planning via recurrent spline optimization. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 7730–7736. IEEE (2021)

27. Yan, Y., Mao, Y., Li, B.: Second: sparsely embedded convolutional detection. *Sensors* **18**(10), 3337 (2018)
28. Zhang, Z., Liniger, A., Dai, D., Yu, F., Van Gool, L.: End-to-end urban driving by imitating a reinforcement learning coach. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15222–15232 (2021)